SOFTWARE ENGINEERING / TESTING SUBMISSION

Voice Assistant for Multimedia Controls

Consolidated test reports

ESP32-WROOM | Offline voice command recognition | Team 3

Document control	Details
Prepared for	Dhanush S S - Integration Lead (INT-02)
Course testing window	2-6 November 2026
Recorded test execution	7 November 2026, 01:50 IST
Edition	Submission presentation, 7 November 2026. Existing measured evidence retained.
Application baseline	91d5a74d815a2daff04afd896e4f135877d552b9
Validation status	Automated host suites passed; final system acceptance incomplete.

Four reports. One evidence-backed submission.

Unit testing, system validation, code coverage and branch coverage are provided in full. Requirement ownership, limitations, reproducible commands and linked execution evidence accompany the results.

Submission overview

What the evidence establishes

Measure	Verified result
C++ automated tests	93/93 passed: 87 unit/contract and 6 integration cases
Optional Python prototype	10/10 test methods passed
C++ implementation line coverage	662/691 = 95.80%
C++ implementation branch coverage	457/518 = 88.22%
Python statement coverage	50/120 = 41.67%
Python branch coverage	14/36 = 38.89%

Coverage applies to the explicitly listed source scope. It does not establish full-product coverage, real speech recognition, real Bluetooth behavior or independent acceptance. Instrumented reruns repeat existing tests and are not counted twice.

Submission checklist

Requested deliverable	Location and outcome
Unit Test Report	Report 1: suite counts, normal/error behavior, exclusions and evidence
System Validation Report	Report 2: all 25 requirement-level STP checks, owners and remaining validation
Code Coverage Report	Report 3: per-file line/statement results and scope exclusions
Branch Coverage Report	Report 4: per-file decision outcomes, unexecuted paths and limitations
Repository documentation	Markdown reports and raw evidence already published in docs/test-reports/2026-11-07/ through PR #49

Reading order and decision

Read the four reports in the order above. They are also supplied as separate PDFs. Raw logs, coverage JSON, annotated results and the reproducible runner remain in the repository; this PDF is the readable presentation of that evidence.

Final acceptance and release freeze are pending. The missing full ESP32 build configuration, actual audio/model integration and physical acceptance tests must be completed and independently reviewed. No acceptance signatures, device results or earlier execution dates have been fabricated.

Unit Test Report

Project: Voice Assistant for Multimedia Controls - Team 3 **Prepared for:** Dhanush S S, Integration Lead **Execution date:** 07 November 2026, 01:50 IST (UTC+05:30) **Application source commit:** `91d5a74d815a2daff04afd896e4f135877d552b9` **Test mode:** author-run automated host tests; independent acceptance pending.

The course testing window shown in the brief is **2-6 November**. These tests were actually run on **7 November 2026 IST**; results are not backdated. UTC timestamps may fall on 6 November. Report date and evidence date are distinct from the course deadline.

Result

PASS for the executed host suites: PlatformIO native **93/93**, independently rebuilt instrumentation run **93/93**, and optional Python prototype **10/10**. The instrumentation run repeats the same 93 existing cases; it is not 93 new tests. Distinct automated test cases/methods reported: **103** (93 C++ + 10 Python). Subtests within Python methods are not counted as extra methods.

The C++ suite contains **87 unit/contract cases and 6 downstream integration cases**. The integration cases are reported separately from unit acceptance.

Executed suites

Existing suite	Test methods/cases	Failures	Classification
test_DR_01_contracts	3	0	Host unit/contract
test_INT_01	6	0	Host integration
test_VAMC_F_005	12	0	Host unit/contract
test_VAMC_F_006	12	0	Host unit/contract
test_VAMC_F_007	5	0	Host unit/contract
test_VAMC_F_008	6	0	Host unit/contract
test_VAMC_F_009	4	0	Host unit/contract
test_VAMC_F_010	6	0	Host unit/contract
test_VAMC_F_011	9	0	Host unit/contract
test_VAMC_F_012	5	0	Host unit/contract
test_VAMC_F_013	3	0	Host unit/contract
test_VAMC_F_014	5	0	Host unit/contract
test_VAMC_SR_002	5	0	Host unit/contract
test_VAMC_SR_003	6	0	Host unit/contract
test_reset_button	6	0	Host unit/contract

Python: `companion/laptop_prototype/tests/test_prototype.py` - 10 methods, 0 failures. It uses mocks/fakes for audio, transcription and media dispatch; actual Windows key injection, serial recording and Whisper inference were not run.

What was exercised

• Seven-command text matching/normalization, phrase variants, ambiguity and rejection.

• HID mapping, send policy, disconnected link, bounded retry/backoff and feedback.

• Provisioning start/input validation, copied credentials, timeout and protected-save errors.

• Portable discovery/passkey policy, reset coordinator and five-second BOOT hold detector.

• Circular-log behavior and foundation contracts.

• Real native OpenSSL credential round trips, wrong passphrase, tampering, fresh nonce, erase, malformed records and storage failures. ESP32 mbedTLS/NVS execution is untested.

• Downstream gate > mapping > fake BLE transport > feedback integration, covering all seven actions, rejection, recognition failure, disconnect, retry and exhaustion.

• Python classifier/mapper, dry-run CLI, rejection, platform guard and cleanup under injected transcription failure.

Missing tests and limits

No on-board wake-word engine, microphone capture, tinyML inference, real authenticated BLE/HID behavior, GPIO output, actual flash inspection, OTA or battery test ran. An empty/fake adapter and a passing text test cannot establish those features. The actual target PlatformIO build failed due to missing configuration; see the build output. No ESP32 serial port was detected.

No coverage threshold was added and no tests were altered to inflate results. Cross-testers must independently run the linked STP cases and record A/F themselves.

Scope and acceptance boundary

The approved project uses ESP32-WROOM and offline fixed-vocabulary recognition. No live cloud API, microphone, flash operation or Bluetooth/media action was performed by this report run. Passing host tests is not completion of the physical SRS/STP acceptance criteria. This report does not set RTM A/F statuses, close issues, complete INT-02, create a release tag or freeze development.

The locally revised SRS/SAD/STP/backlog implement Dhanush's offline redesign; some revisions are still unpublished and remote source text remains inconsistent. The exact source-document hashes are in source-baselines.json. Harsh/DOC-01 must reconcile and publish those baselines before final acceptance. Legacy `TC-ASR-*` IDs are retained; their names do not authorize cloud recognition.

Reproduction and evidence

From the repository root, with PlatformIO installed:

```
pio test -d firmware -e native
python3 -m pip install coverage==7.17.2
python3 scripts/measure_coverage.py --output /tmp/vamc-report-rerun
```

The coverage runner currently supports **macOS** with Xcode command-line tools, Homebrew `openssl@3` and Unity installed by the PlatformIO test run. It compiles the same existing C++ tests and library sources with Apple Clang/LLVM 21 at `-O0`, using `-fprofile-instr-generate -fcoverage-mapping`. Test/Unity code is not instrumented. All 12

portable `.cpp` objects are linked into every suite; the 15 raw profiles are merged once, and one binary provides their shared mappings. This avoids counting shared code 15 times. The runner then invokes Python coverage with `--branch`, including all six prototype source files and excluding its tests. It records tool versions, commands, source commit and suite counts.

- Run manifest and exact commands
- Coverage scope inventory
- PlatformIO native test output
- C++ coverage summary
- C++ annotated lines and branch counts
- C++ machine-readable coverage
- Python test output
- Python coverage summary
- Python machine-readable coverage
- Python annotated HTML

Method reference: Clang source-based coverage documentation. Execution evidence, rather than documentation examples, supplies every result here.

System Validation Report

Project: Voice Assistant for Multimedia Controls - Team 3 **Prepared for:** Dhanush S S, Integration Lead **Execution date:** 07 November 2026, 01:50 IST (UTC+05:30) **Application source commit:** `91d5a74d815a2daff04afd896e4f135877d552b9` **Test mode:** author-run automated host tests; independent acceptance pending.

The course testing window shown in the brief is **2-6 November**. These tests were actually run on **7 November 2026 IST**; results are not backdated. UTC timestamps may fall on 6 November. Report date and evidence date are distinct from the course deadline.

Validation verdict

FINAL SYSTEM VALIDATION INCOMPLETE - not ready for acceptance/freeze.

The 93 C++ host cases and 10 optional Python cases passed. Six of the C++ cases exercise downstream integration using a fake BLE transport. **No full wake-word > microphone > offline WROOM recognition > Bluetooth HID > host action trial was executed in this run.**

Of the **25 requirement-level STP cases**, **0 received new independent final acceptance in this run**. This is an evidence boundary, not an assertion that no earlier physical experiment ever occurred. Prior results require identifiable test records before they can count toward acceptance. The 103 automated cases are a different granularity and must not be reported as 103 STP cases passed.

Environment and blockers observed

• Host: macOS/arm64, Python 3.13, Apple Clang/LLVM 21; exact versions in manifest.

• USB scan showed Bluetooth/debug/OnePlus ports and **no ESP32 serial device**.

• `pio run -d firmware -e esp32dev` failed with `UnknownEnvNamesError: Unknown environment names 'esp32dev'. Valid names are 'native'.`

• Main contains an inert Arduino entry point, wake-word/recognizer placeholders and portable/fake boundaries; the final target model and physical adapters still need integration. No completed on-board recognition engine is assumed.

• No live microphone, Bluetooth host media event, Windows dispatch, flash/security inspection, battery run or OTA update was performed. No device was flashed.

• No paid API or cloud speech service was used. The optional Python local-model path is development scope, not a replacement for WROOM acceptance.

Existing earlier Arduino compile smoke results are not rerun results here and are not substituted for the missing configured full target build.

Requirement traceability - current validation evidence

Requirement / STP test	Owner > independent tester	Acceptance criterion	Evidence in this run / remaining validation
VAMC-F-001 / TC-WW-01	G S Dheekshith > Harsh Mannan	Wake word spoken at normal volume within 2 m triggers the listening state.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required

Requirement / STP test	Owner > independent tester	Acceptance criterion	Evidence in this run / remaining validation
VAMC-F-002 / TC-WW-02	G S Dheekshith > Harsh Mannan	Capture microphone audio for up to 5 s in bounded reusable frames; do not require retaining a complete five-second waveform in RAM.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-F-003 / TC-WW-03	G S Dheekshith > Harsh Mannan	Pressing mute disables audio capture until unmuted; LED shows mute state.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-F-004 / TC-ASR-01	Harsh Mannan > Chiranthan Shankar	A validated target-board model returns a supported command class within 3 s after utterance end with internet unavailable; no audio is transmitted externally.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-F-005 / TC-ASR-02	Harsh Mannan > Chiranthan Shankar	Every phrase in the approved test set maps to the correct one of volume up, volume down, pause, resume, play, next, or previous; variants are included in the fixed model vocabulary.	**Not accepted in this run.** 12 text-matcher tests; trained audio/model variants unverified
VAMC-F-006 / TC-ASR-03	Harsh Mannan > Chiranthan Shankar	An unrecognized/ambiguous test phrase triggers error feedback and no command ID is sent to command mapping.	**Not accepted in this run.** 12 gate tests + downstream rejection tests; actual spoken rejection unverified
VAMC-F-007 / TC-CMD-01	Chiranthan Shankar > Dhanush S S	All 7 commands independently verified against the host media player.	**Not accepted in this run.** 5 mapping tests + downstream seven-command test; host actions unverified
VAMC-F-008 / TC-CMD-02	Chiranthan Shankar > Dhanush S S	Host receives and executes the key event within 2 s of recognition.	**Not accepted in this run.** 6 send-policy tests; real host execution/timing unverified
VAMC-F-009 / TC-CMD-03	Chiranthan Shankar > Dhanush S S	3 retries with backoff, then an error is indicated.	**Not accepted in this run.** 4 retry tests + downstream retry cases; real reconnect unverified
VAMC-F-010 / TC-CMD-04	Chiranthan Shankar > Dhanush S S	LED/buzzer confirms success within 1 s of execution.	**Not accepted in this run.** 6 feedback tests; GPIO/buzzer and real timing unverified
VAMC-F-011 / TC-CONN-01	Dhanush S S > G S Dheekshith	Device completes guided Wi-Fi provisioning within 5 min; voice recognition and BLE media commands remain independent of internet availability.	**Not accepted in this run.** 9 coordinator tests; real portal/Wi-Fi/timing unverified
VAMC-F-012 / TC-CONN-02	Dhanush S S > G S Dheekshith	Device appears in the host's Bluetooth list and pairs successfully.	**Not accepted in this run.** 5 discovery/pairing tests; radio/host pairing unverified
VAMC-F-013 / TC-CONN-03	Dhanush S S > G S Dheekshith	Long-press clears stored Wi-Fi/BLE data and restarts provisioning.	**Not accepted in this run.** 3 coordinator + 6 button cases; real erasure/reprovisioning unverified

Requirement / STP test	Owner > independent tester	Acceptance criterion	Evidence in this run / remaining validation
VAMC-F-014 / TC-DIAG-01	Chiranthan Shankar > Dhanush S S	Last N events retrievable over the serial/debug console; no audio or credentials logged.	**Not accepted in this run.** 5 logging tests; actual device serial retrieval unverified
VAMC-F-015 / TC-DIAG-02	Chiranthan Shankar > Dhanush S S	New firmware image applied remotely without physical access.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-NF-001 / TC-PERF-01	G S Dheekshith > Harsh Mannan	90th percentile wake-word-to-action latency <= 3 s over 50 trials.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-NF-002 / TC-ACC-01	Harsh Mannan > Chiranthan Shankar	Accuracy >= 90% over 100 test utterances in a quiet indoor environment.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-NF-003 / TC-UX-01	Harsh Mannan > G S Dheekshith	Average setup time <= 5 min across 5 first-time users following the setup guide.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-NF-004 / TC-POWER-01	G S Dheekshith > Harsh Mannan	Continuous idle-listening operation >= 8 h on a 2000 mAh battery.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-NF-005 / TC-MAINT-01	G S Dheekshith > Dhanush S S	Code review confirms module boundaries; one module replaced in isolation without edits to others.	**Not accepted in this run.** Public interfaces present; prescribed replacement review not executed
VAMC-SR-001 / TC-SEC-01	Harsh Mannan > Chiranthan Shankar	With internet unavailable, supported commands remain recognizable; network inspection shows no command audio or transcript is sent externally.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-SR-002 / TC-SEC-02	Dhanush S S > G S Dheekshith	Pairing on a supporting host requires passkey/numeric confirmation, not 'Just Works'.	**Not accepted in this run.** 5 policy tests; actual host passkey flow unverified
VAMC-SR-003 / TC-SEC-03	Dhanush S S > G S Dheekshith	Flash inspection after provisioning shows no plaintext credentials.	**Not accepted in this run.** 6 native OpenSSL vault tests; ESP32 flash/mbedTLS unverified
VAMC-SR-004 / TC-SEC-04	G S Dheekshith > Harsh Mannan	Memory/flash inspection shows no retained audio after the session ends.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required
VAMC-SR-005 / TC-SEC-05	Chiranthan Shankar > Dhanush S S	Unsigned/tampered image is rejected before it is applied.	**Not accepted in this run.** No target acceptance execution; implementation/hardware/test environment still required

NF-005 is a review/replacement requirement rather than a physical measurement; its full prescribed review was not executed by this run. All other listed device/system acceptance results remain unexecuted here. The table does not overwrite the project's RTM status codes.

Integration and release evidence

• INT-01: six host integration cases passed; seven command mappings, no-send rejection, recognition error, disconnected host, retry recovery/exhaustion tested. Full spoken-command/host evidence remains pending.

• INT-02: partial regression evidence only. All planned stories are not accepted; no release tag/freeze is created or justified by these reports.

• VID-01/VID-02: no new working-product videos were recorded/uploaded in this run.

• GitHub issue/PR snapshots are preserved in the evidence folder. Merged code and green checks do not imply original hardware or cross-test acceptance.

Defects / blockers and completion plan

Item	Responsible owner	Next action / evidence needed
Target build environment missing	Dhanush + Dheekshith	Approve/configure reproducible WROOM build; run it in CI
Microphone/wake path not validated	Dheekshith; Harsh cross-tests	Approved mic/pins/phrase/engine, real capture/wake/mute/cleanup records
Offline recognizer not validated	Harsh; Chiranthan cross-tests	Target model fit, actual command/variant/error/privacy and accuracy results
Real HID and feedback	Chiranthan; Dhanush cross-tests	Seven observed host actions, timing/retry, GPIO and serial evidence
Setup/pairing/storage/reset adapters	Dhanush; Dheekshith cross-tests	Real portal/passkey, flash inspection, boot unlock timing, actual BOOT reset
Performance/usability/power	Assigned owners in matrix	50 latency trials, 100 utterances/SNR, 5 users, 8-hour battery run
OTA/signature tests	Chiranthan; Dhanush cross-tests	Approved keys/source/partitions; valid update and invalid-image rejection
Source baseline reconciliation	Harsh/DOC-01 + Dhanush review	Publish approved offline revisions, resolve stale source descriptions
Uncovered source paths	Respective module owners	Meaningful fault/protocol/platform tests, then rerun coverage

Do not backfill actual results, claim a missing experiment passed, or change dates to match the earlier course window. Record independent results when run.

Scope and acceptance boundary

The approved project uses ESP32-WROOM and offline fixed-vocabulary recognition. No live cloud API, microphone, flash operation or Bluetooth/media action was performed by this report run. Passing host tests is not completion of the physical SRS/STP acceptance criteria. This report does not set RTM A/F statuses, close issues, complete INT-02, create a release tag or freeze development.

The locally revised SRS/SAD/STP/backlog implement Dhanush's offline redesign; some revisions are still unpublished and remote source text remains inconsistent. The exact source-document hashes are in source-baselines.json. Harsh/DOC-01 must reconcile and publish those baselines before final acceptance. Legacy `TC-ASR-*` IDs are retained; their names do not authorize cloud recognition.

Reproduction and evidence

From the repository root, with PlatformIO installed:

```
pio test -d firmware -e native
python3 -m pip install coverage==7.17.2
python3 scripts/measure_coverage.py --output /tmp/vamc-report-rerun
```

The coverage runner currently supports **macOS** with Xcode command-line tools, Homebrew `openssl@3` and Unity installed by the PlatformIO test run. It compiles the same existing C++ tests and library sources with Apple Clang/LLVM 21 at `-O0`, using `-fprofile-instr-generate -fcoverage-mapping`. Test/Unity code is not instrumented. All 12 portable `.cpp` objects are linked into every suite; the 15 raw profiles are merged once, and one binary provides their shared mappings. This avoids counting shared code 15 times. The runner then invokes Python coverage with `--branch`, including all six prototype source files and excluding its tests. It records tool versions, commands, source commit and suite counts.

- Run manifest and exact commands
- Coverage scope inventory
- PlatformIO native test output
- C++ coverage summary
- C++ annotated lines and branch counts
- C++ machine-readable coverage
- Python test output
- Python coverage summary
- Python machine-readable coverage
- Python annotated HTML

Method reference: Clang source-based coverage documentation. Execution evidence, rather than documentation examples, supplies every result here.

Code Coverage Report

Project: Voice Assistant for Multimedia Controls - Team 3 **Prepared for:** Dhanush S S, Integration Lead **Execution date:** 07 November 2026, 01:50 IST (UTC+05:30) **Application source commit:** `91d5a74d815a2daff04afd896e4f135877d552b9` **Test mode:** author-run automated host tests; independent acceptance pending.

The course testing window shown in the brief is **2-6 November**. These tests were actually run on **7 November 2026 IST**; results are not backdated. UTC timestamps may fall on 6 November. Report date and evidence date are distinct from the course deadline.

Measured results

Scope	Primary code metric	Result
12 portable C++ implementation files	Executed source lines / executable source lines	**662 / 691 = 95.80%**
Same C++ scope	Executed functions	**55 / 61 = 90.16%**
Same C++ scope	Executed coverage regions	**617 / 644 = 95.81%**
Six optional Python prototype files	Executed statements / statements	**50 / 120 = 41.67%**

"Code coverage" here means line coverage for C++, and statement coverage for Python. These are not SRS requirement-acceptance percentages. Results are not averaged across the two different runtimes or presented as 100% project coverage.

Explicit C++ denominator

All **12 existing portable implementation `.cpp` files** under `firmware/lib/` are compiled and included, including files not directly referenced by a suite. Only those `.cpp` mappings enter the percentages. **Headers/inline-only logic** (including `reset_button.h`), `firmware/src` hardware adapters/entry point, examples, tests, generated code, Unity, OpenSSL/framework internals and tooling are excluded. ESP32-only conditional code (notably mbedTLS) is not compiled in this host run. Unimplemented features have no code denominator and are not implicitly "covered". This is not whole-product or target-firmware coverage.

The scope inventory lists compiled files and excluded headers. The BOOT detector has six passing tests but no percentage in this measurement. Report it as tested, not as measured coverage.

C++ file results

Portable implementation file	Covered / executable lines	Line coverage
asr_client/command_gate.cpp	34 / 34	100.00%
asr_client/command_matcher.cpp	139 / 139	100.00%
ble_hid/ble_hid.cpp	57 / 69	82.61%
command_map/command_map.cpp	113 / 115	98.26%
connectivity/authenticated_pairing.cpp	32 / 32	100.00%

Portable implementation file	Covered / executable lines	Line coverage
connectivity/ble_discovery.cpp	24 / 24	100.00%
connectivity/credential_vault.cpp	77 / 78	98.72%
connectivity/factory_reset.cpp	11 / 11	100.00%
connectivity/wifi_provisioning.cpp	52 / 52	100.00%
diagnostics_ota/diagnostics_ota.cpp	48 / 48	100.00%
feedback/feedback.cpp	64 / 78	82.05%
pipeline/command_dispatch.cpp	11 / 11	100.00%

There are **29 uncovered executable C++ lines** in scope. See the annotated evidence for exact locations. BLE stub/default paths and feedback paths account for most uncovered lines. Allocation/crypto-error paths also need targeted fault-injection consideration. This report adds no tests in teammates' modules.

Optional Python prototype - full source scope

Prototype file	Covered / statements	Statement coverage
command_classifier.py	7 / 7	100.00%
command_mapper.py	3 / 3	100.00%
main.py	30 / 34	88.24%
media_controller.py	10 / 25	40.00%
receive_audio.py	0 / 43	0.00%
speech_to_text.py	0 / 8	0.00%

`receive_audio.py` and `speech_to_text.py` are **0%**, not omitted. Serial transport, actual local model loading/transcription and real Windows media execution still need suitable tests/environments. Python coverage's displayed combined figure is **41.03%** because it combines statements and branch outcomes (64/156); that is different from statement-only **41.67%** and is not substituted for it.

No minimum line/statement target was specified for this reporting task. The measured result is reported without inventing a pass threshold. Improvements should target meaningful uncovered behavior, not merely higher percentages.

Scope and acceptance boundary

The approved project uses ESP32-WROOM and offline fixed-vocabulary recognition. No live cloud API, microphone, flash operation or Bluetooth/media action was performed by this report run. Passing host tests is not completion of the physical SRS/STP acceptance criteria. This report does not set RTM A/F statuses, close issues, complete INT-02, create a release tag or freeze development.

The locally revised SRS/SAD/STP/backlog implement Dhanush's offline redesign; some revisions are still unpublished and remote source text remains inconsistent. The exact source-document hashes are in source-baselines.json. Harsh/DOC-01 must reconcile and publish those baselines before final acceptance. Legacy `TC-ASR-*` IDs are retained; their names do not authorize cloud recognition.

Reproduction and evidence

From the repository root, with PlatformIO installed:

```
pio test -d firmware -e native
python3 -m pip install coverage==7.17.2
python3 scripts/measure_coverage.py --output /tmp/vamc-report-rerun
```

The coverage runner currently supports **macOS** with Xcode command-line tools, Homebrew `openssl@3` and Unity installed by the PlatformIO test run. It compiles the same existing C++ tests and library sources with Apple Clang/LLVM 21 at `-O0`, using `-fprofile-instr-generate -fcoverage-mapping`. Test/Unity code is not instrumented. All 12 portable `.cpp` objects are linked into every suite; the 15 raw profiles are merged once, and one binary provides their shared mappings. This avoids counting shared code 15 times. The runner then invokes Python coverage with `--branch`, including all six prototype source files and excluding its tests. It records tool versions, commands, source commit and suite counts.

- Run manifest and exact commands
- Coverage scope inventory
- PlatformIO native test output
- C++ coverage summary
- C++ annotated lines and branch counts
- C++ machine-readable coverage
- Python test output
- Python coverage summary
- Python machine-readable coverage
- Python annotated HTML

Method reference: Clang source-based coverage documentation. Execution evidence, rather than documentation examples, supplies every result here.

Branch Coverage Report

Project: Voice Assistant for Multimedia Controls - Team 3 **Prepared for:** Dhanush S S, Integration Lead **Execution date:** 07 November 2026, 01:50 IST (UTC+05:30) **Application source commit:** `91d5a74d815a2daff04afd896e4f135877d552b9` **Test mode:** author-run automated host tests; independent acceptance pending.

The course testing window shown in the brief is **2-6 November**. These tests were actually run on **7 November 2026 IST**; results are not backdated. UTC timestamps may fall on 6 November. Report date and evidence date are distinct from the course deadline.

Measured results

Scope	Covered branch outcomes / measured outcomes	Result
12 portable C++ implementation files	**457 / 518**	**88.22%**
Six optional Python prototype files	**14 / 36**	**38.89%**

Uncovered outcomes: **61 C++**, **22 Python**. Python additionally reports **4 partially covered branch points**. Its partial-branch count is not the number of missing outcomes. A branch outcome is a measured direction of a decision, not a test case or an independently accepted requirement.

LLVM measures instrumented source decision outcomes, including relevant short-circuit conditions. Python coverage.py measures its source branch arcs. They use different models; no pooled percentage is meaningful. **MC/DC was not enabled, and no MC/DC or complete-path-coverage claim is made.**

Explicit C++ denominator

All **12 existing portable implementation `.cpp` files** under `firmware/lib/` are compiled and included, including files not directly referenced by a suite. Only those `.cpp` mappings enter the percentages. **Headers/inline-only logic** (including `reset_button.h`), `firmware/src` hardware adapters/entry point, examples, tests, generated code, Unity, OpenSSL/framework internals and tooling are excluded. ESP32-only conditional code (notably mbedTLS) is not compiled in this host run. Unimplemented features have no code denominator and are not implicitly "covered". This is not whole-product or target-firmware coverage.

The scope inventory lists compiled files and excluded headers. The BOOT detector has six passing tests but no percentage in this measurement. Report it as tested, not as measured coverage.

C++ file results

Portable implementation file	Covered / branch outcomes	Branch coverage
asr_client/command_gate.cpp	13 / 14	92.86%
asr_client/command_matcher.cpp	110 / 114	96.49%
ble_hid/ble_hid.cpp	23 / 30	76.67%
command_map/command_map.cpp	125 / 126	99.21%
connectivity/authenticated_pairing.cpp	16 / 18	88.89%

Portable implementation file	Covered / branch outcomes	Branch coverage
connectivity/ble_discovery.cpp	10 / 12	83.33%
connectivity/credential_vault.cpp	73 / 100	73.00%
connectivity/factory_reset.cpp	6 / 6	100.00%
connectivity/wifi_provisioning.cpp	31 / 36	86.11%
diagnostics_ota/diagnostics_ota.cpp	15 / 22	68.18%
feedback/feedback.cpp	31 / 36	86.11%
pipeline/command_dispatch.cpp	4 / 4	100.00%

Optional Python file results

Prototype file	Covered / branch outcomes	Branch coverage
command_classifier.py	2 / 2	100.00%
command_mapper.py	0 / 0	N/A - no measured branches
main.py	7 / 10	70.00%
media_controller.py	5 / 12	41.67%
receive_audio.py	0 / 10	0.00%
speech_to_text.py	0 / 2	0.00%

Files with no measured branches are N/A, not automatically 100% branch-tested.

Remaining branch work

• Credential vault: 27 missing outcomes; inspect input, crypto/allocation and storage-failure decisions. Native results exclude the ESP32 mbedTLS path.

• BLE sender: 7 missing outcomes, including default/stub and edge-case paths.

• Diagnostics: 7 missing outcomes; inspect configured log edge cases.

• Other missing C++ outcomes are shown individually in the annotated evidence.

• Python serial capture and transcription are wholly unexecuted. Add suitable protocol/error tests and separately record real serial/model tests.

• Real Windows key dispatch requires appropriate platform tests; mock-based mapping tests do not exercise the operating-system calls.

Allocation-failure/error branches need deliberate, meaningful fault injection; do not mark them covered because the normal path passed. No arbitrary branch threshold or new acceptance criterion is introduced by this report.

Scope and acceptance boundary

The approved project uses ESP32-WROOM and offline fixed-vocabulary recognition. No live cloud API, microphone, flash operation or Bluetooth/media action was performed by this report run. Passing host tests is not completion of the physical SRS/STP acceptance criteria. This report does not set RTM A/F statuses, close issues, complete INT-02, create a release tag or freeze development.

The locally revised SRS/SAD/STP/backlog implement Dhanush's offline redesign; some revisions are still unpublished and remote source text remains inconsistent. The exact source-document hashes are in source-baselines.json. Harsh/DOC-01 must reconcile and publish those baselines before final acceptance. Legacy `TC-ASR-*` IDs are retained; their names do not authorize cloud recognition.

Reproduction and evidence

From the repository root, with PlatformIO installed:

```
pio test -d firmware -e native
python3 -m pip install coverage==7.17.2
python3 scripts/measure_coverage.py --output /tmp/vamc-report-rerun
```

The coverage runner currently supports **macOS** with Xcode command-line tools, Homebrew `openssl@3` and Unity installed by the PlatformIO test run. It compiles the same existing C++ tests and library sources with Apple Clang/LLVM 21 at `-O0`, using `-fprofile-instr-generate -fcoverage-mapping`. Test/Unity code is not instrumented. All 12 portable `.cpp` objects are linked into every suite; the 15 raw profiles are merged once, and one binary provides their shared mappings. This avoids counting shared code 15 times. The runner then invokes Python coverage with `--branch`, including all six prototype source files and excluding its tests. It records tool versions, commands, source commit and suite counts.

- Run manifest and exact commands
- Coverage scope inventory
- PlatformIO native test output
- C++ coverage summary
- C++ annotated lines and branch counts
- C++ machine-readable coverage
- Python test output
- Python coverage summary
- Python machine-readable coverage
- Python annotated HTML

Method reference: Clang source-based coverage documentation. Execution evidence, rather than documentation examples, supplies every result here.